

Week2 day5 cod codes:

```
CREATE OR REPLACE DATABASE LOGISTICS_LAB;
```

```
CREATE OR REPLACE SCHEMA DATA;
```

```
USE DATABASE LOGISTICS_LAB;
```

```
USE SCHEMA DATA;
```

```
CREATE OR REPLACE TABLE DELIVERIES(
```

```
ORDER_ID STRING,
```

```
ORDER_DATE DATE,
```

```
CITY STRING,
```

```
VEHICLE_TYPE STRING,
```

```
DISTANCE_KM NUMBER(8,2),
```

```
DELIVERY_TIME_MIN INT,
```

```
IS_ONTIME STRING,
```

```
DRIVER_RATING NUMBER(3,1)
```

```
);
```

```
--TOTAL ORDERS AND OVERALL ON-TIME RATE
```

```
select
```

```
count(*) AS TOTAL_ORDERS,
```

```
ROUND(100.0*AVG(CASE WHEN IS_ONTIME='Yes' THEN 1.0 ELSE 0.0 END),1)||'%' AS  
ONTIME_PERCENTAGE
```

```
FROM DELIVERIES;
```

```
--PERFORMANCE BY CITY
```

```
SELECT
```

```
CITY,
```

```
COUNT(*) AS TOTAL_DELIVERIES,

ROUND(AVG(DELIVERY_TIME_MIN),1) AS AVG_MINUTES,

ROUND(100.0*AVG(CASE WHEN IS_ONTIME='Yes' THEN 1.0 ELSE 0.0 END),1) || '%' AS
ONTIME_PCT

FROM DELIVERIES

GROUP BY CITY

ORDER BY ONTIME_PCT DESC;

--VEHICLE TYPE PERFORMANCE

SELECT

VEHICLE_TYPE,

ROUND(AVG(DELIVERY_TIME_MIN),1) AS AVG_MINUTES,

ROUND(AVG(DISTANCE_KM),1) AS AVG_DISTANCE_KM,

ROUND(100.0*AVG(CASE WHEN IS_ONTIME='Yes' THEN 1.0 ELSE 0.0 END),1) || '%' AS
ONTIME_PCT

FROM DELIVERIES

GROUP BY VEHICLE_TYPE

ORDER BY AVG_MINUTES;

--TOP 10 BEST-RATED DELIVERIES

SELECT

ORDER_ID,DRIVER_RATING,CITY,VEHICLE_TYPE,IS_ONTIME

FROM DELIVERIES

ORDER BY DRIVER_RATING DESC

LIMIT 10;

--DAILY DELIVERY VOLUME

SELECT

ORDER_DATE,

COUNT(*) AS DELIVERIES_PER_DAY
```

```
FROM DELIVERIES
GROUP BY ORDER_DATE
ORDER BY ORDER_DATE;
```

—streamlit code

```
import streamlit as st
import pandas as pd
from snowflake.snowpark.context import get_active_session
```

```
# -----
```

```
# Snowflake Session
```

```
# -----
```

```
session = get_active_session()
```

```
# -----
```

```
# Page Config
```

```
# -----
```

```
st.set_page_config(
    page_title="Logistics Dashboard",
    layout="wide"
)
```

```
st.title(" Logistics Delivery Dashboard")
```

```
# -----
```

```

# Load Data

# -----

query = """
SELECT
    ORDER_ID,
    ORDER_DATE,
    CITY,
    VEHICLE_TYPE,
    DISTANCE_KM,
    DELIVERY_TIME_MIN,
    IS_ONTIME,
    DRIVER_RATING
FROM LOGISTICS_LAB.DATA.DELIVERIES
"""

df = session.sql(query).to_pandas()

# -----

# Filters

# -----

st.sidebar.header("Filters")

city_filter = st.sidebar.multiselect(
    "Select City",
    options=df["CITY"].unique(),
    default=df["CITY"].unique()

```

)

```
vehicle_filter = st.sidebar.multiselect(
    "Select Vehicle Type",
    options=df["VEHICLE_TYPE"].unique(),
    default=df["VEHICLE_TYPE"].unique()
)
```

```
filtered_df = df[
    (df["CITY"].isin(city_filter)) &
    (df["VEHICLE_TYPE"].isin(vehicle_filter))
]
```

```
# -----
```

```
# KPIs
```

```
# -----
```

```
total_orders = len(filtered_df)
```

```
ontime_percentage = (
    100 * (filtered_df["IS_ONTIME"] == "Yes").mean()
)
```

```
avg_delivery_time = filtered_df["DELIVERY_TIME_MIN"].mean()
```

```
avg_rating = filtered_df["DRIVER_RATING"].mean()
```

```
col1, col2, col3, col4 = st.columns(4)
```

```
col1.metric(  
    "Total Orders",  
    f"{total_orders:.,}"  
)
```

```
col2.metric(  
    "On-Time Delivery",  
    f"{ontime_percentage:.1f}%"  
)
```

```
col3.metric(  
    "Avg Delivery Time",  
    f"{avg_delivery_time:.1f} min"  
)
```

```
col4.metric(  
    "Avg Driver Rating",  
    f"{avg_rating:.1f} ☆"  
)
```

```
st.divider()
```

```
# -----
```

```
# Delivery Performance by City
```

```
# -----
```

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    st.subheader(" Deliveries by City")
```

```
    city_deliveries = (  
        filtered_df.groupby("CITY")  
        .size()  
        .sort_values(ascending=False)  
    )
```

```
    st.bar_chart(city_deliveries)
```

```
with col2:
```

```
    st.subheader("🕒 Average Delivery Time by City")
```

```
    city_time = (  
        filtered_df.groupby("CITY")["DELIVERY_TIME_MIN"]  
        .mean()  
        .round(1)  
        .sort_values()  
    )
```

```

st.bar_chart(city_time)

# -----
# Vehicle Performance
# -----

st.subheader(" Vehicle Performance")

vehicle_performance = (
    filtered_df.groupby("VEHICLE_TYPE")
    .agg(
        AVG_DELIVERY_TIME=("DELIVERY_TIME_MIN", "mean"),
        AVG_DISTANCE=("DISTANCE_KM", "mean"),
        AVG_RATING=("DRIVER_RATING", "mean")
    )
    .round(1)
)

st.dataframe(
    vehicle_performance,
    use_container_width=True
)

# -----
# On-Time Delivery by City
# -----

st.subheader(" On-Time Delivery by City")

```



```
city_ontime = (  
    filtered_df.assign(  
        ONTIME=filtered_df["IS_ONTIME"] == "Yes"  
    )  
    .groupby("CITY")["ONTIME"]  
    .mean()  
    .mul(100)  
    .round(1)  
    .sort_values(ascending=False)  
)
```

```
st.bar_chart(city_ontime)
```

```
# -----
```

```
# Daily Delivery Volume
```

```
# -----
```

```
st.subheader(" Daily Delivery Volume")
```

```
daily_delivery = (  
    filtered_df.groupby("ORDER_DATE")  
    .size()  
    .reset_index(name="DELIVERIES")  
    .sort_values("ORDER_DATE")  
)
```

```
st.line_chart(  
    daily_delivery,  
    x="ORDER_DATE",  
    y="DELIVERIES"  
)
```

```
# -----  
# Distance vs Delivery Time  
# -----
```

```
st.subheader("📊 Distance vs Delivery Time")
```

```
distance_time = filtered_df[  
    ["DISTANCE_KM", "DELIVERY_TIME_MIN"]  
]
```

```
st.scatter_chart(  
    distance_time,  
    x="DISTANCE_KM",  
    y="DELIVERY_TIME_MIN"  
)
```

```
# -----  
# Top 10 Best Rated Drivers  
# -----
```

```
st.subheader("📊 Top 10 Best Rated Deliveries")
```

```

topRated = (
    filtered_df[
        [
            "ORDER_ID",
            "DRIVER_RATING",
            "CITY",
            "VEHICLE_TYPE",
            "IS_ONTIME"
        ]
    ]
    .sort_values("DRIVER_RATING", ascending=False)
    .head(10)
)

```

```

st.dataframe(
    topRated,
    use_container_width=True
)

```

```

# -----

```

```

# Sales / Delivery Details

```

```

# -----

```

```

st.subheader("📦 Delivery Details")

```

```

st.dataframe(
    filtered_df,

```

use_container_width=True

)